# Deploy a Python FastMCP Server to Azure App Service (Web App)

Step-by-step walkthrough with screenshots

| | |
|---|---|
| **Tool stack** | VS Code, Python 3.12, FastMCP 4.1.3, Azure CLI (no Docker) |
| **Azure service** | Azure App Service - Linux Web App, Free (F1) plan |
| **Web app / resource group / region** | fastmcp-webapp-8e5zs / fastmcp-webapp-rg / centralindia |
| **MCP endpoint** | https://fastmcp-webapp-8e5zs.azurewebsites.net/mcp |
| **Transport** | Streamable HTTP |
| **Captured on** | 21 October 2026 |

Every screenshot in this walkthrough comes from a real run on Windows: terminal screenshots show the actual output of each command in the VS Code PowerShell terminal, and portal screenshots were taken from portal.azure.com right after the deployment. Subscription ID, tenant ID and e-mail address are masked (<subscription-id>, <tenant-id>, <your-email> or blurred). Very long outputs are shortened with an '... lines omitted ...' marker.

# What we are building

The same FastMCP server with one tool, **add(a, b)**, hosted on **Azure App Service** - Azure's classic platform for web apps. App Service provides a Linux machine with Python already installed; we upload our code, it installs requirements.txt and runs our start command. **server.py is exactly the same file as in the Container Apps walkthrough.**

```
MCP Client  (Microsoft Foundry / MCP Inspector / test_client.py)
     |
     |  HTTPS  -  Streamable HTTP  -  /mcp
     v
Azure App Service front end  (https://<app>.azurewebsites.net)
     |  forwards to port 8000 inside the app
     v
Linux Web App (Python 3.12)  ->  python server.py  ->  add(a, b)
```

## How this differs from the Container Apps version

• **No Dockerfile** - App Service's built-in Python runtime builds and runs the code.
• We create an **App Service plan** (the machine size/price) and a **web app** on it.
• The **startup command** `python server.py` tells App Service how to start our server.
• Code is uploaded as a **zip** and built in Azure (`SCM_DO_BUILD_DURING_DEPLOYMENT`).

## Prerequisites

• An Azure subscription • Azure CLI • Python 3.10+ and VS Code - see **Before you start**. Docker is **not** needed.

# Steps at a glance

Steps 1-2 happen on your laptop; steps 3-12 talk to Azure. The complete text of every file is in **Appendix A** for copying.

| # | Step | Where | What you do | Why |
|---|---|---|---|---|
| 1 | Create the project | VS Code | server.py, requirements.txt, test_client.py | The MCP server code; @mcp.tool turns add() into a tool. |
| 2 | Test locally | VS Code terminal | venv, pip install, python server.py, python test_client.py | Find bugs before deploying. |
| 3 | Prepare Azure | VS Code terminal | az login, az account show, register Microsoft.Web | Sign in, confirm the subscription, enable App Service (one-time). |
| 4 | Resource group | VS Code terminal | az group create --name $RG --location $LOC | One folder for every resource of this project. |
| 5 | App Service plan | VS Code terminal | az appservice plan create --name $PLAN --sku F1 --is-linux | The Linux machine (size and price) the app runs on. F1 is free. |
| 6 | Web app | VS Code terminal | az webapp create ... --runtime PYTHON:3.12 --startup-file "python server.py" | Creates the app with Python and tells it how to start our server. |
| 7 | App settings | VS Code terminal | SCM_DO_BUILD_DURING_DEPLOYMENT=true, WEBSITES_PORT=8000 | Install requirements.txt during deploy; send traffic to port 8000. |
| 8 | Deploy | VS Code terminal | Compress-Archive ... app.zip; az webapp deploy --src-path app.zip --type zip | Uploads the code; Azure builds and starts it. |
| 9 | Get the URL | VS Code terminal + portal | az webapp show --query defaultHostName | Public hostname; add /mcp for the MCP endpoint. |
| 10 | Logs | VS Code terminal | az webapp log config / az webapp log tail | See the server start and handle requests. |
| 11 | Test and connect | VS Code terminal + Foundry | python test_client.py <URL>; add MCP tool in Foundry | Prove it works over the internet; use it from an AI agent. |
| 12 | Clean up | VS Code terminal / portal | az group delete --name $RG | Stop all charges. |

# Before you start: install the tools

Do this once on each student laptop (Windows 10/11). Open **PowerShell** and run the `winget` commands, or download the installers from the links.

## 1. Python 3.12

```
winget install -e --id Python.Python.3.12
```

Or download from **python.org/downloads**. In the installer tick **Add python.exe to PATH**.

## 2. Visual Studio Code

```
winget install -e --id Microsoft.VisualStudioCode
```

Or download from **code.visualstudio.com**. In VS Code open Extensions (Ctrl+Shift+X) and install the **Python** extension from Microsoft.

## 3. Azure CLI

```
winget install -e --id Microsoft.AzureCLI
```

Or download the MSI from **aka.ms/installazurecliwindows**.

## 4. Check the installation

**Close and reopen** VS Code (so it picks up the new PATH), open a terminal (Terminal > New Terminal) and run:

```
python --version
az version
```

Both commands must print a version number. If `python` opens the Microsoft Store instead, reinstall Python with *Add python.exe to PATH* ticked.

## 5. An Azure subscription

Each student needs an Azure subscription: a free account (**azure.microsoft.com/free**), **Azure for Students** (no credit card, via a school e-mail), or a subscription provided by the institution. Running everything in one shared subscription works only if every student uses different names (see the naming step).

Docker Desktop is **not** required for any of these walkthroughs - Azure builds everything in the cloud.

# Step 1: Create the project in VS Code

Create a folder named **fastmcp-webapp**, open it in VS Code (File > Open Folder) and add the files below (copy them from **Appendix A**).




Figure 1.1 - Project folder contents

## requirements.txt

```
fastmcp==4.1.3
uvicorn==0.54.0
```

Figure 1.2 - requirements.txt

## server.py

Unchanged from the Container Apps version. `host="0.0.0.0"` and `port=8000` matter: App Service forwards traffic to port 8000 inside the app.

```python
from fastmcp import FastMCP

mcp = FastMCP("Azure MCP Server")


@mcp.tool
def add(a: int, b: int) -> int:
    """Add two numbers."""
    return a + b


if __name__ == "__main__":
    mcp.run(
        transport="http",
        host="0.0.0.0",
        port=8000,
        path="/mcp"
    )
```

Figure 1.3 - server.py

## test_client.py (used to verify the server)

```
import asyncio
```

```
import sys

from fastmcp import Client

URL = sys.argv[1] if len(sys.argv) > 1 else "http://localhost:8000/mcp"


async def main():
    async with Client(URL) as client:
        tools = await client.list_tools()
        print("Connected to:", URL)
        print("Tools available:", [tool.name for tool in tools])

        result = await client.call_tool("add", {"a": 5, "b": 7})
        print("add(5, 7) =", result.data)


asyncio.run(main())
```

## Step 2: Test locally in VS Code

Before touching Azure, prove the server works on your laptop. Open the VS Code terminal (Terminal > New Terminal) inside the project folder:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

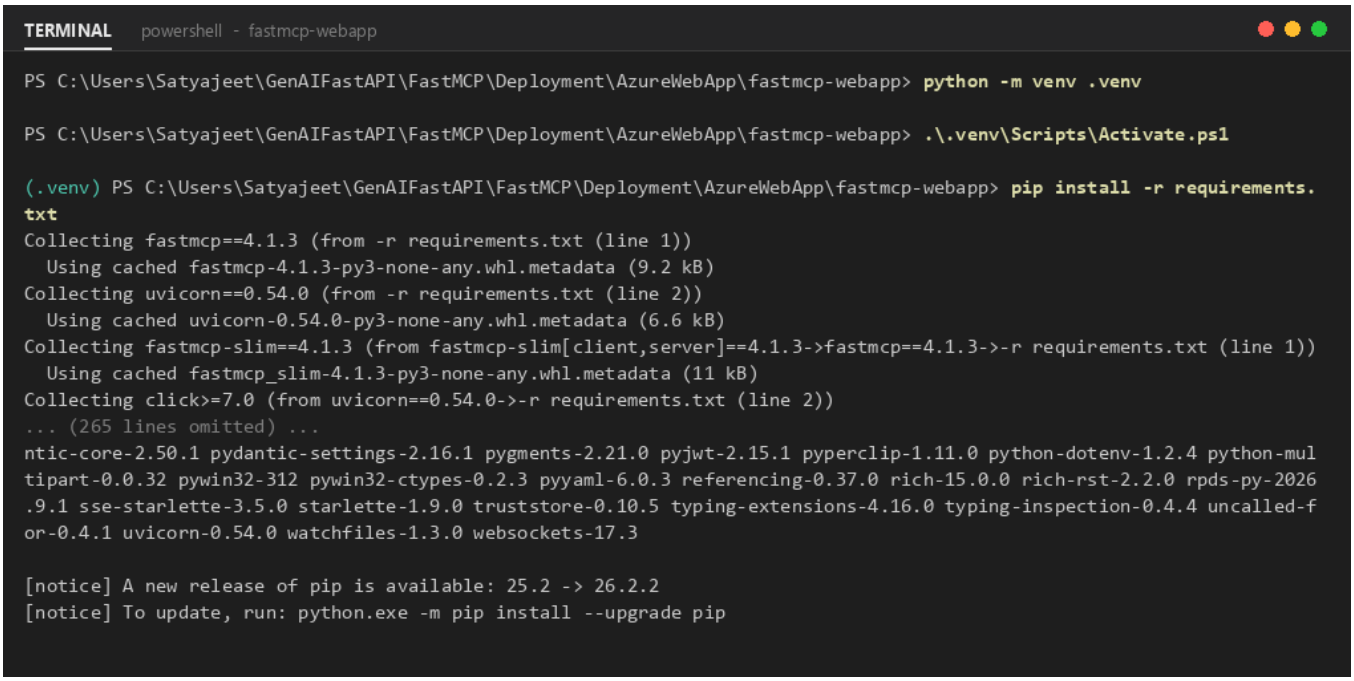



Figure 2.1 - Create the virtual environment, activate it and install dependencies

If activation fails with *running scripts is disabled on this system*, run once: `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`

Start the server and keep this terminal running:

```
python server.py
```

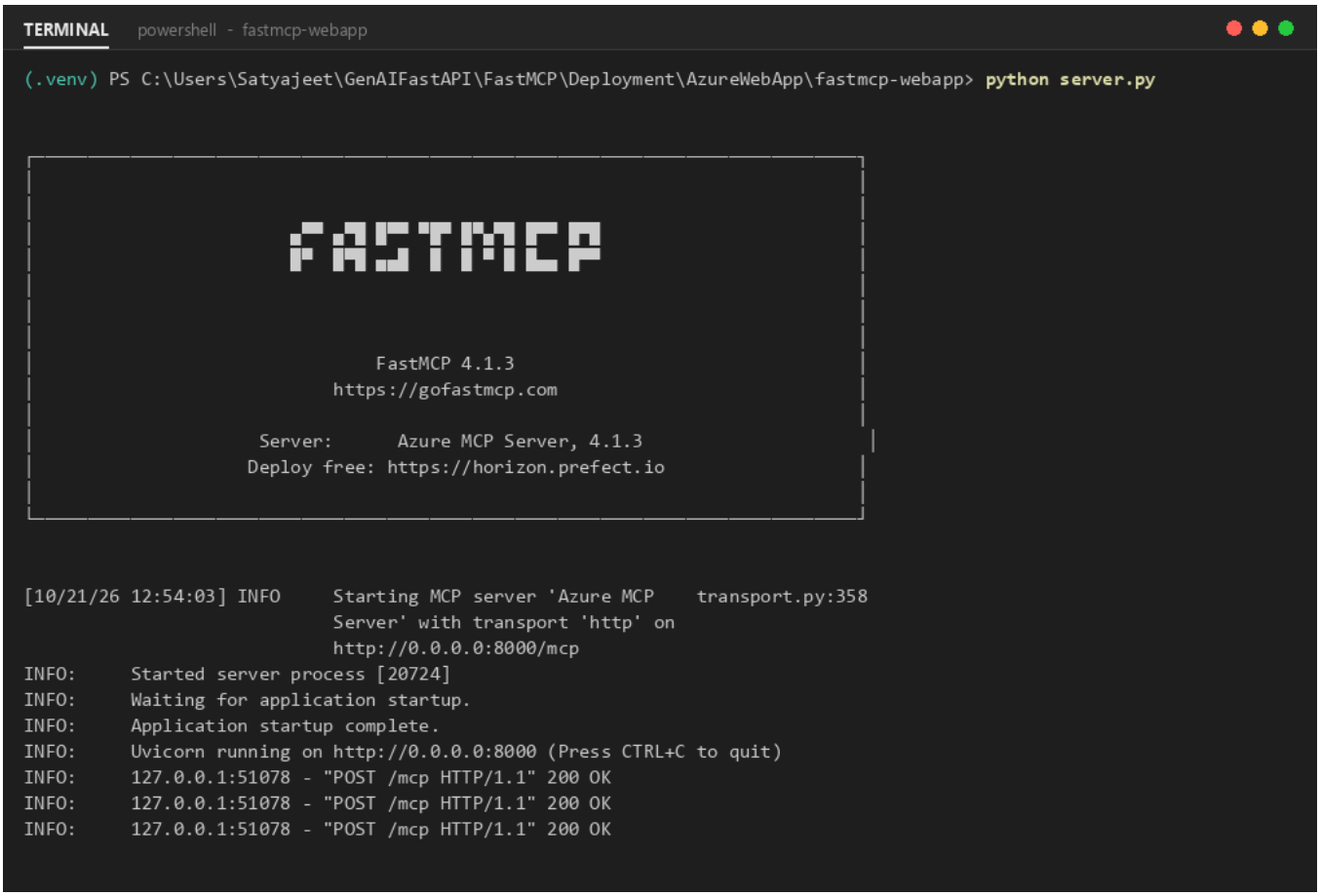



Figure 2.2 - FastMCP server running locally on http://0.0.0.0:8000/mcp

Open a **second** terminal (the + button in the terminal panel), activate the venv again and run the test client:

```
python test_client.py
```

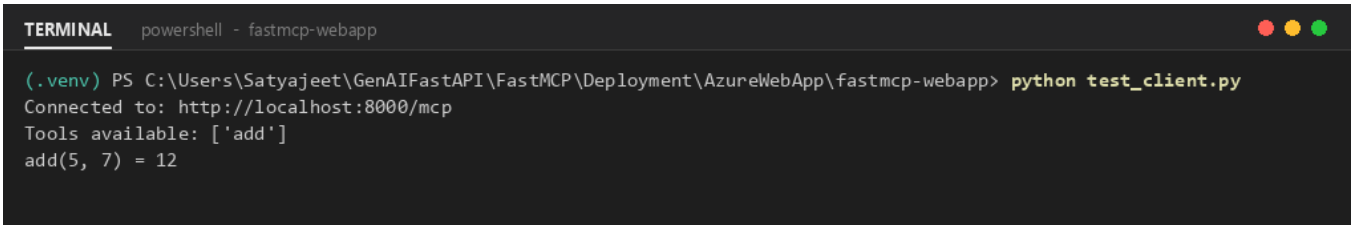



Figure 2.3 - The client discovers the add tool and calls it

Stop the server with **Ctrl+C** before moving on.

# Step 3: Prepare Azure

```
az login
az account show
az provider register --namespace Microsoft.Web
```

`az login` opens a browser to sign in; `az account show` confirms which subscription will be billed.

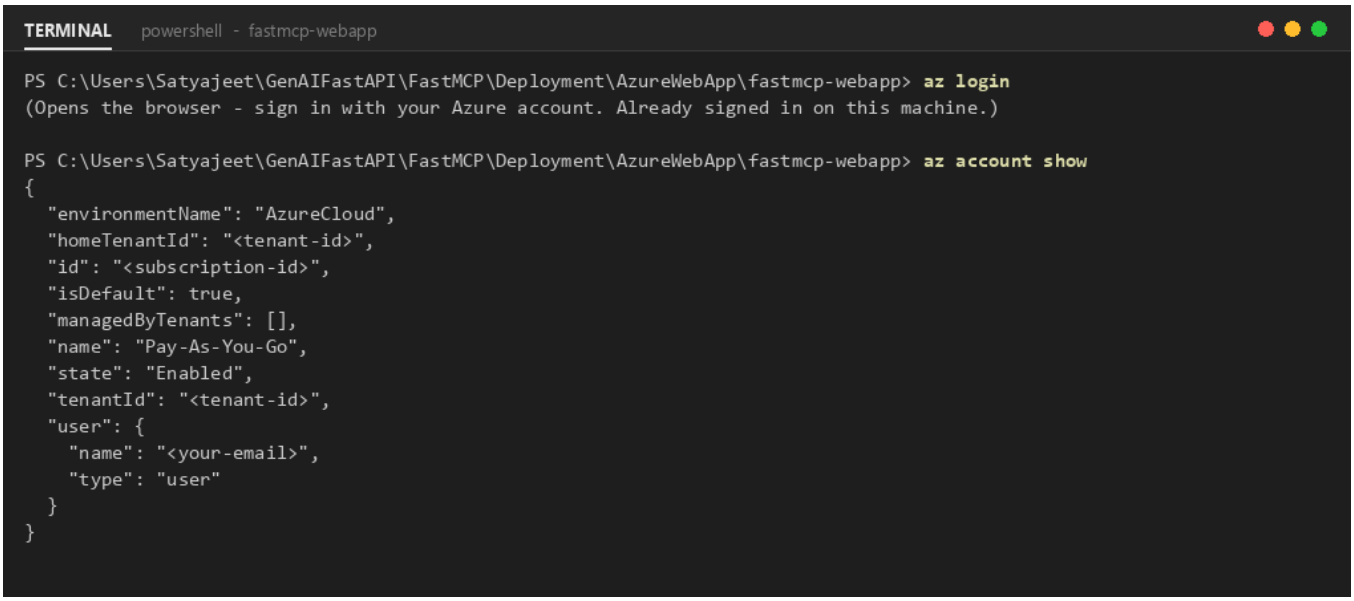



Figure 3.1 - Signed in; active subscription shown (IDs masked)

Register the App Service resource provider (one-time per subscription):

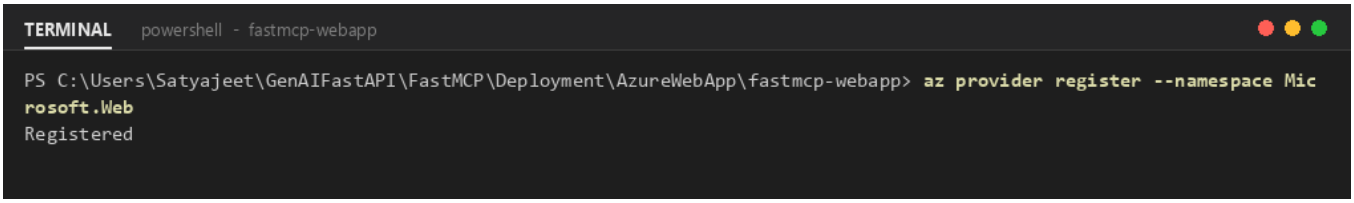



Figure 3.2 - Microsoft.Web provider registered

## Choose your names (do this once)

Some Azure names become part of a public URL, so they must be **unique across all of Azure**. Pick a short **suffix** of your own - lowercase letters and digits, e.g. your initials plus a number - and store all names in PowerShell variables. Every command below uses these variables, so you can copy the commands exactly.

```
$SUFFIX = "ab12"                  # <- change to your own: lowercase letters/digits
$RG     = "fastmcp-webapp-rg"
$LOC    = "centralindia"
$PLAN   = "fastmcp-webapp-plan"
$APP    = "fastmcp-webapp-$SUFFIX"    # becomes https://<name>.azurewebsites.net
```

The web app name may contain letters, digits and dashes (2-60 characters). Variables live only in the current terminal: if you close VS Code or open a new terminal, run this block again before continuing.

*The screenshots show the values used when this guide was made (suffix **8e5zs**); your output will show your own names.*

## Step 4: Create a resource group

A resource group is a folder that holds every resource of this project, so we can see them together and delete them together.

```
az group create --name $RG --location $LOC
```

**Region tip:** this guide uses `centralindia`. Some subscriptions - especially **Azure for Students** - only allow certain regions. If a command fails with *RequestDisallowedByAzure*, *LocationNotAvailableForResourceType* or a policy error, choose another region (for example `eastus`, `westeurope` or `southeastasia`) and use it in **every** command. To list the regions that support this service run `az appservice list-locations --sku F1 --linux-workers-enabled -o table`.

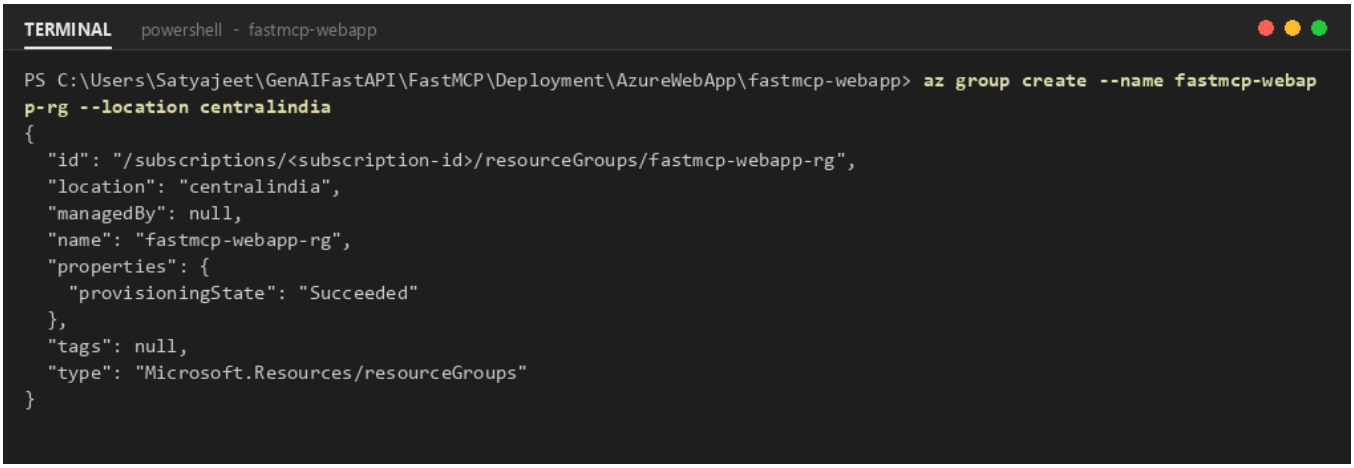



Figure 4.1 - Resource group created

## Step 5: Create an App Service plan

The **plan** is the machine your app runs on: operating system, size and price. `--is-linux` because Python apps run on Linux; `--sku F1` is the **Free** tier.

```
az appservice plan create --name $PLAN --resource-group $RG `
    --location $LOC --sku F1 --is-linux
```

**PowerShell tip:** a backtick ` at the end of a line continues the command on the next line (nothing may follow it, not even a space). You can also type the whole command on one line.

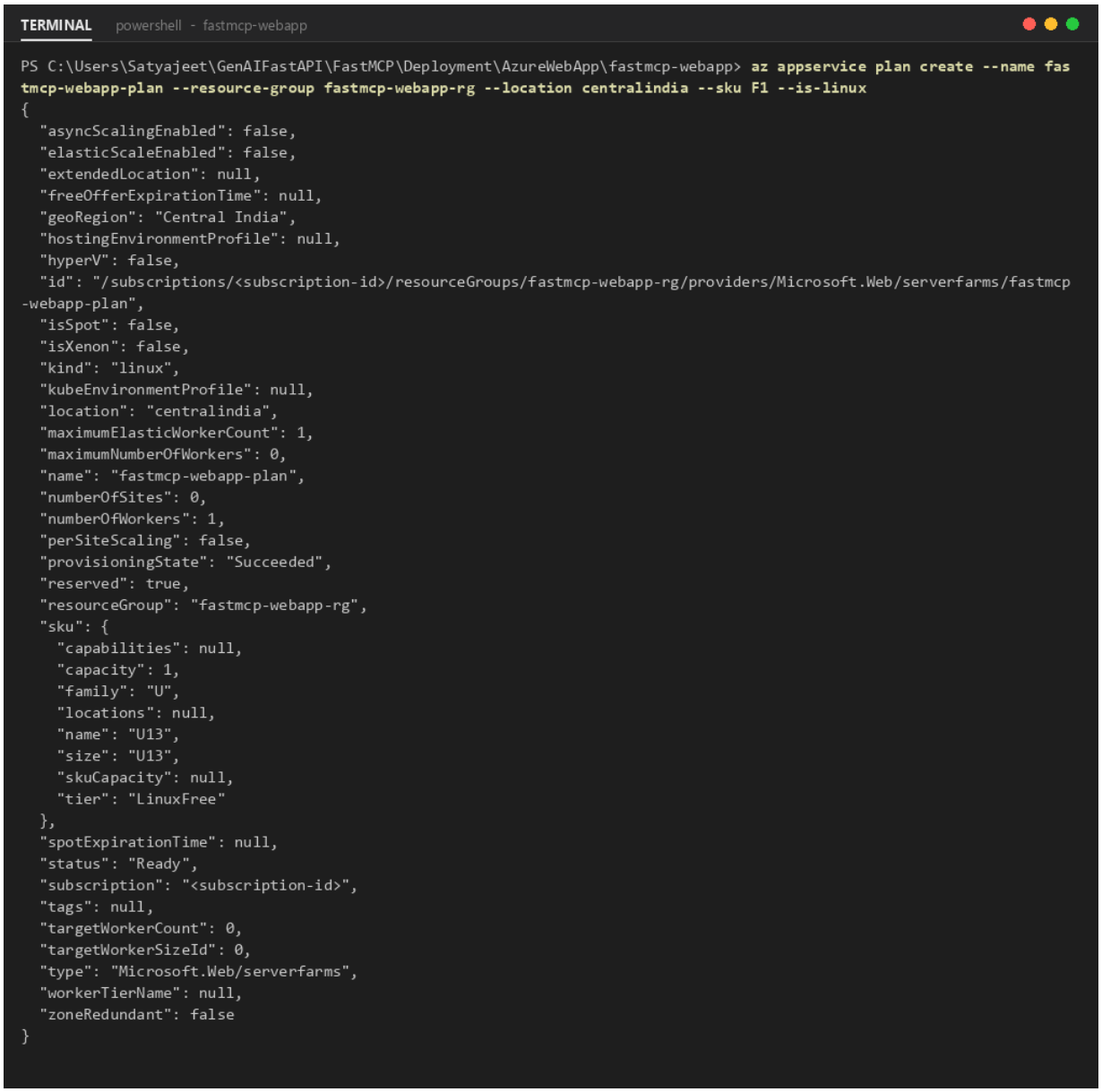



Figure 5.1 - App Service plan created (output shortened)

**Free (F1) limits:** 60 CPU minutes per day, no 'Always On' (the app sleeps after ~20 idle minutes and the next request takes a few seconds to wake it), and only **one free Linux plan per region per subscription**. Fine for a class; use **B1** (Basic) or higher for real use.

# Step 6: Create the web app

Creates the app on our plan with the Python 3.12 runtime. The **startup command** tells App Service to run our server instead of looking for a Django/Flask app. The app name becomes the URL, so it must be globally unique.

```
az webapp create --name $APP --resource-group $RG --plan $PLAN `
    --runtime "PYTHON:3.12" --startup-file "python server.py"
```

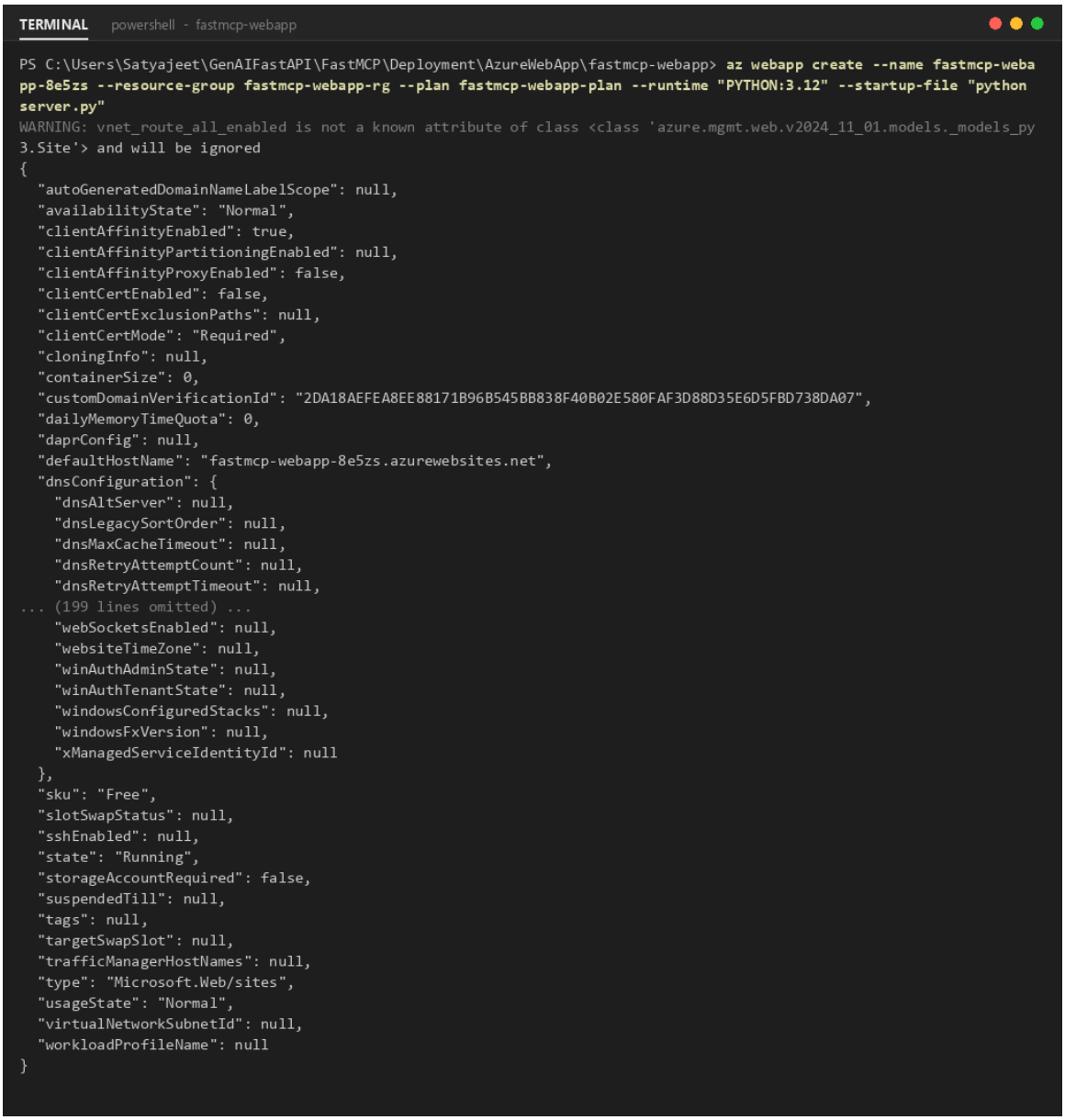



Figure 6.1 - Web app created (output shortened)

# Step 7: Configure app settings

• `SCM_DO_BUILD_DURING_DEPLOYMENT=true` - when we upload a zip, Azure runs `pip install -r requirements.txt` for us.
• `WEBSITES_PORT=8000` - send incoming traffic to port 8000, where server.py listens.

```
az webapp config appsettings set --name $APP --resource-group $RG `
    --settings SCM_DO_BUILD_DURING_DEPLOYMENT=true WEBSITES_PORT=8000
```

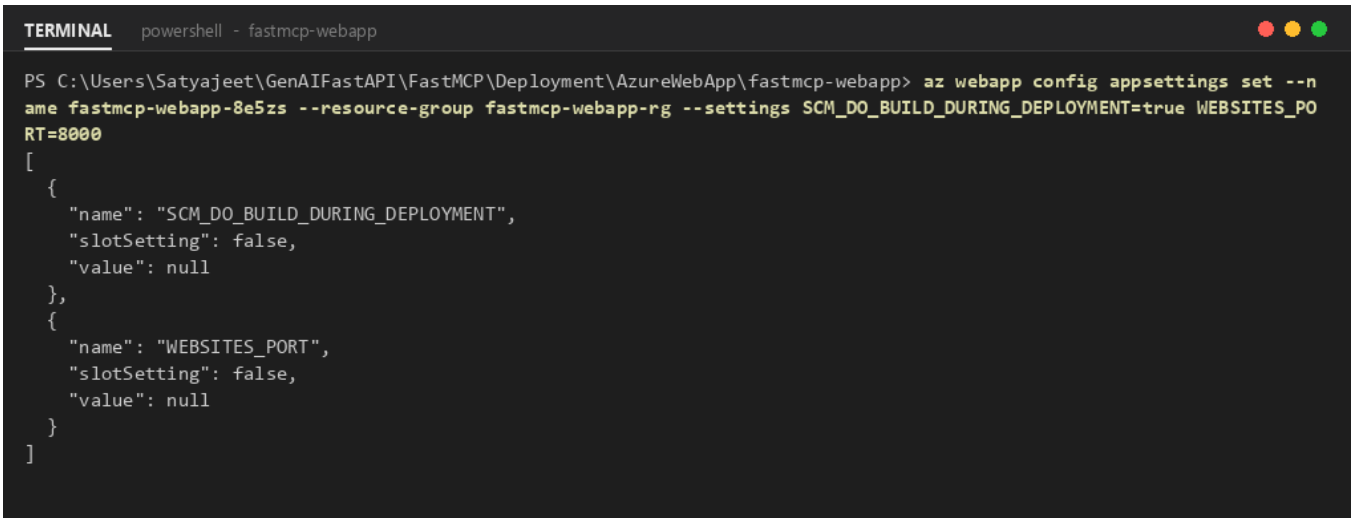



Figure 7.1 - App settings added (the CLI hides values in its output)

# Step 8: Package and deploy the code

Zip only the files the server needs and upload them. Azure builds the app (installs the packages) and starts it with our startup command. This takes 1-3 minutes.

```
Compress-Archive -Path server.py, requirements.txt -DestinationPath app.zip -Force
az webapp deploy --name $APP --resource-group $RG --src-path app.zip --type zip
```

Do **not** zip the .venv folder: it contains Windows packages that will not run on Linux. The build in Azure installs the right ones.

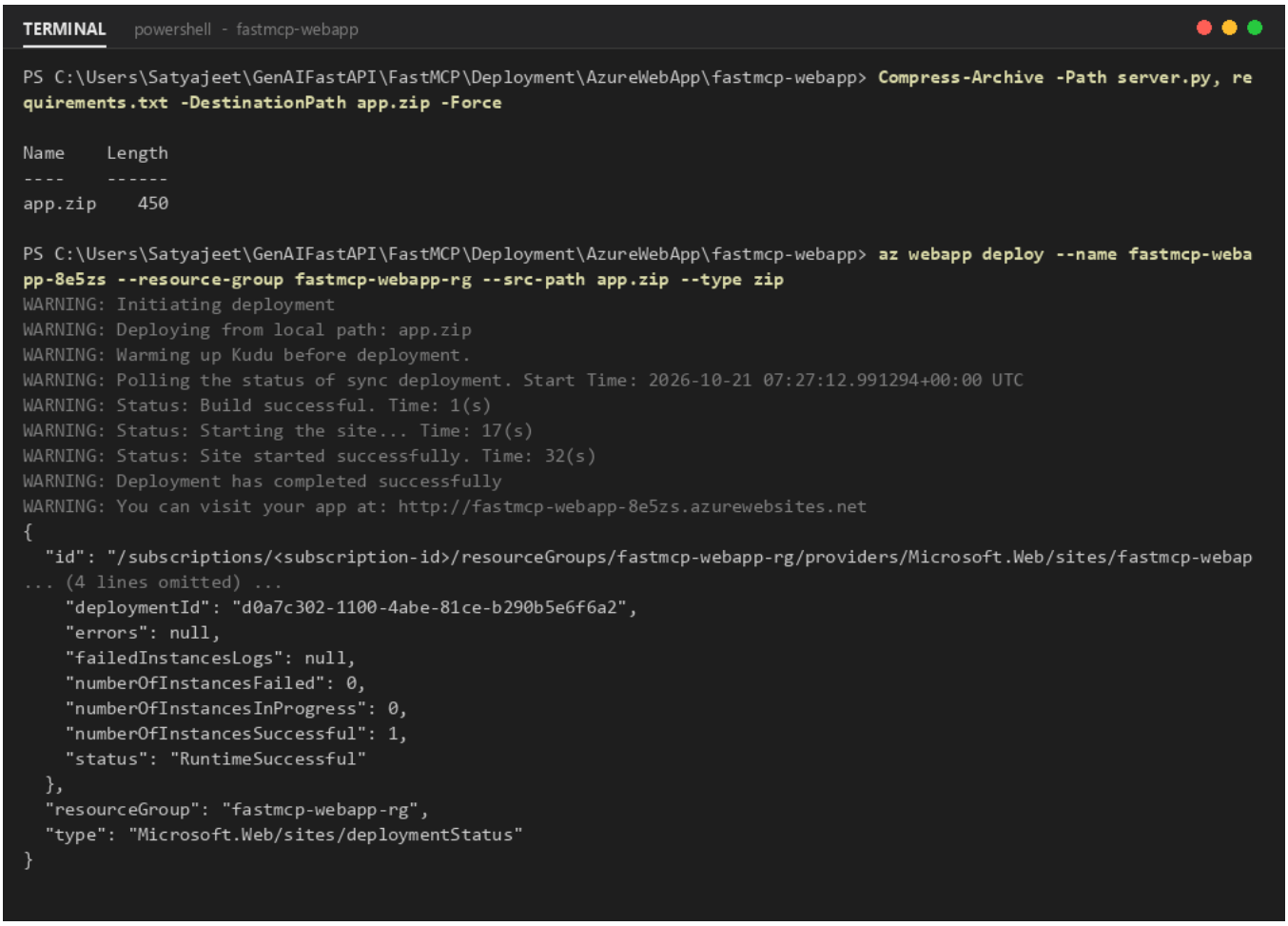



Figure 8.1 - Deployment finished: status RuntimeSuccessful (output shortened)

## Step 9: Get your MCP URL

Ask Azure for the app's hostname and build the MCP URL from it (stored in $URL for the next steps):

```
az webapp show --name $APP --resource-group $RG --query defaultHostName -o tsv
$URL = "https://" + (az webapp show --name $APP --resource-group $RG --query defaultHostName -o tsv) + "/m
$URL
```

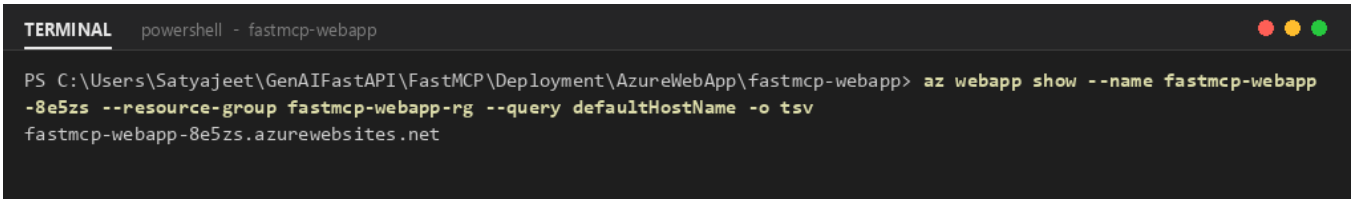



Figure 9.1 - The web app's hostname

The MCP endpoint is **https://** + hostname + **/mcp**. In this run it was:

```
https://fastmcp-webapp-8e5zs.azurewebsites.net/mcp
```

## In the Azure portal: see what was created

Open **portal.azure.com** > **Resource groups** > **fastmcp-webapp-rg**: just two resources - the App Service plan and the web app.

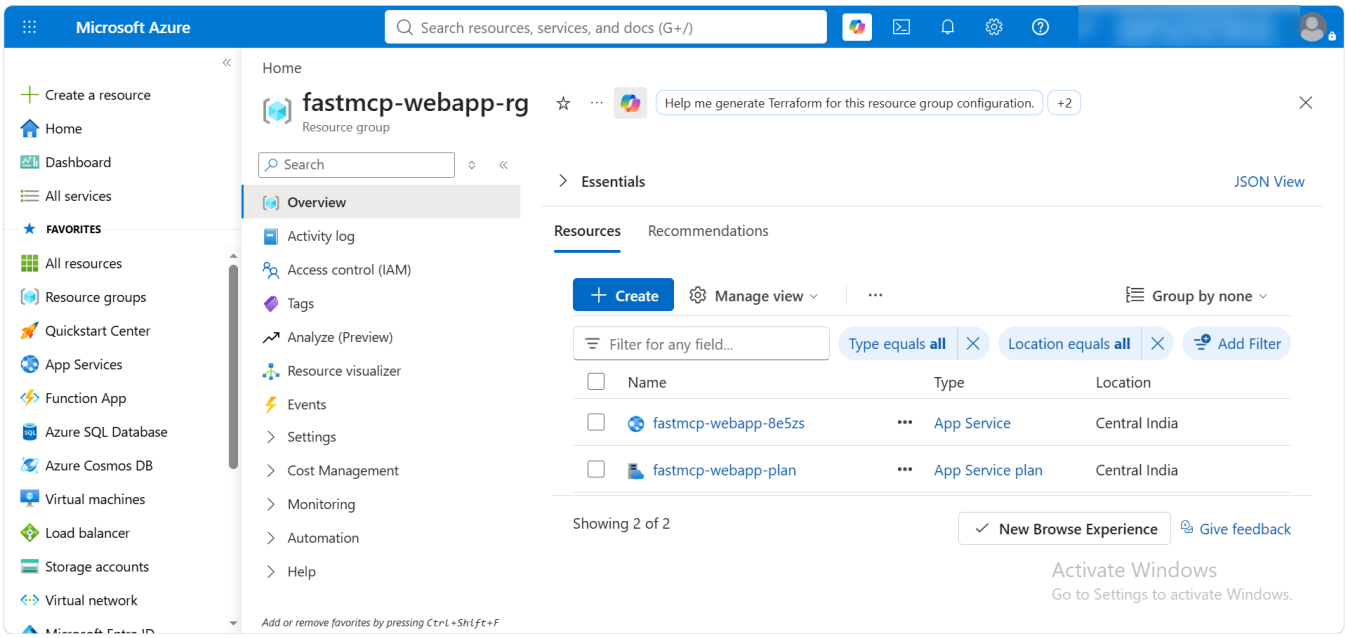



Figure 9.2 - Portal: the resource group

Open the web app > **Overview**: Status *Running*, App Service Plan *fastmcp-webapp-plan (F1)*, Operating System *Linux*, and the **Default domain**.

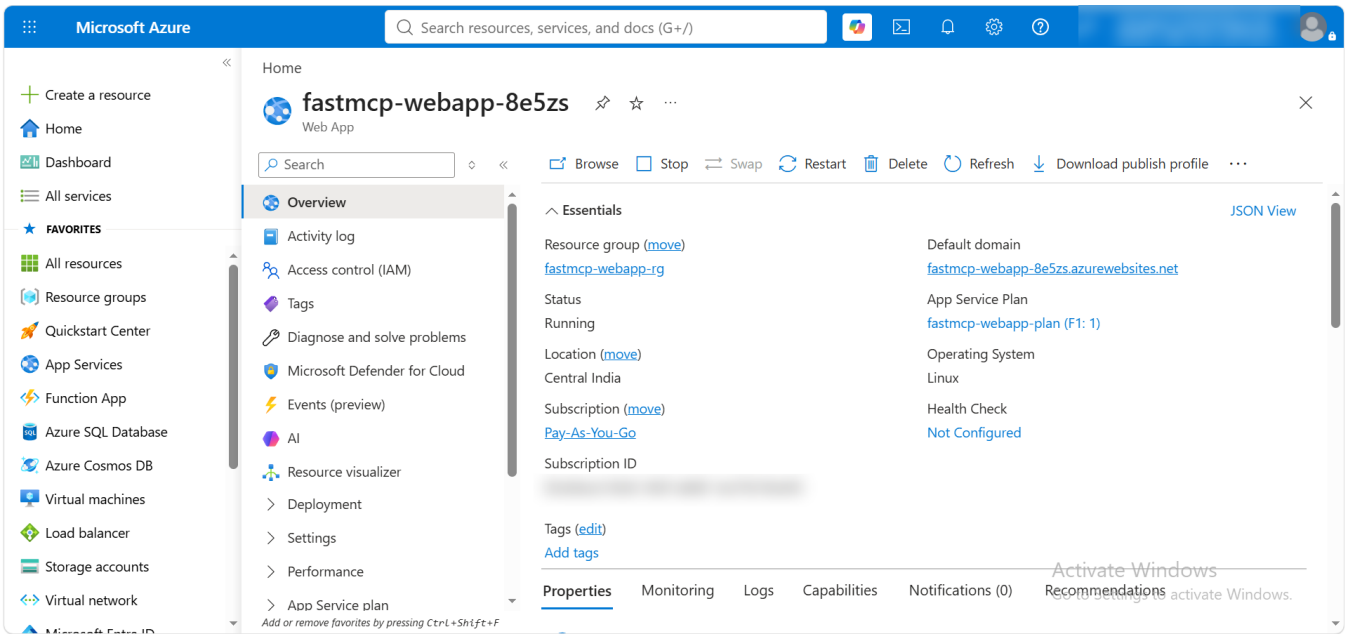



Figure 9.3 - Portal: Web App overview

**Settings** > **Environment variables** shows the app settings from Step 7 (click Show value to see them).

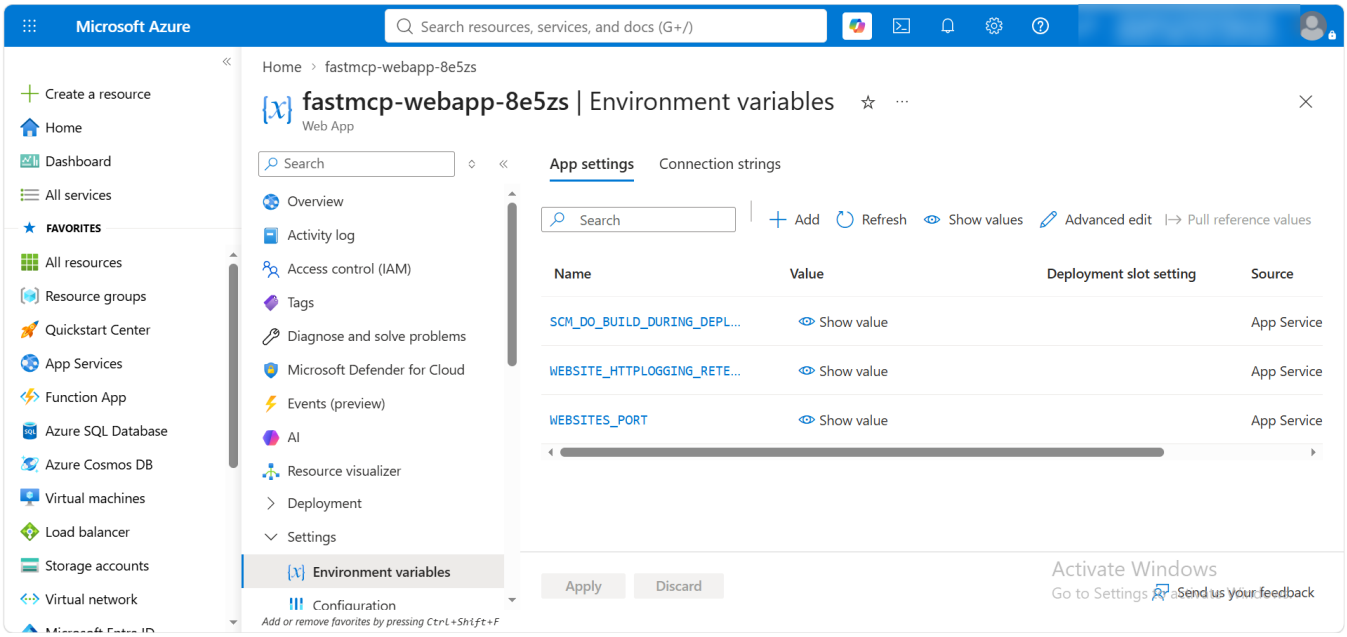



Figure 9.4 - Portal: Environment variables (app settings)

# Step 10: Watch the logs

Turn on container logging, then stream the logs. Run the test client in another terminal while it is streaming: you see the FastMCP server start and the `POST /mcp ... 200 OK` requests. Press **Ctrl+C** to stop streaming.

```
az webapp log config --name $APP --resource-group $RG --docker-container-logging filesystem
az webapp log tail --name $APP --resource-group $RG
```

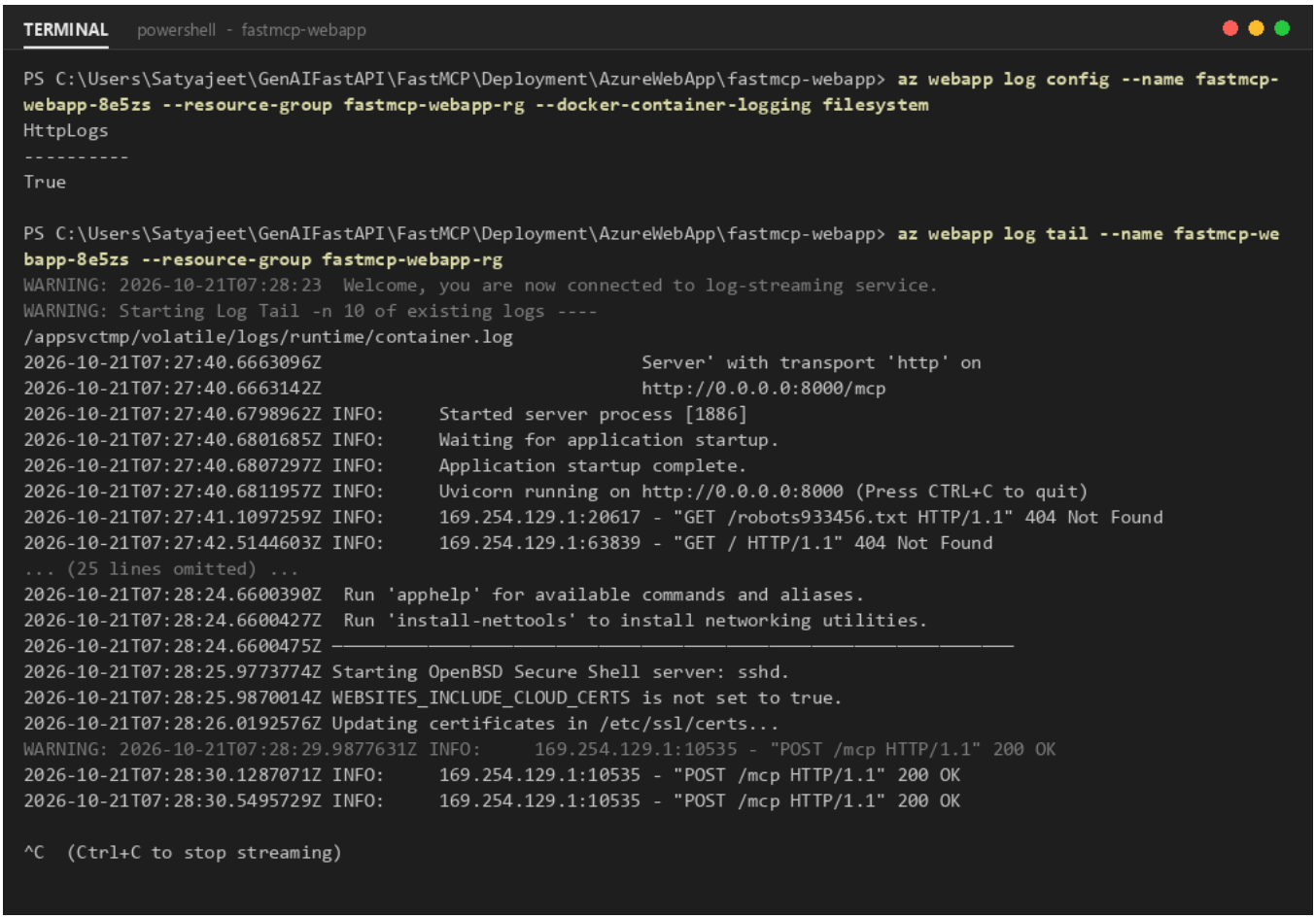



Figure 10.1 - Live logs: server started on 0.0.0.0:8000 and MCP requests returning 200 OK

The portal shows the same stream under **Monitoring** > **Log stream**.

# Step 11: Test the deployed server and connect Microsoft Foundry

Run the same test client from Step 2 against the Azure URL (the venv must be active):

```
python test_client.py $URL
```

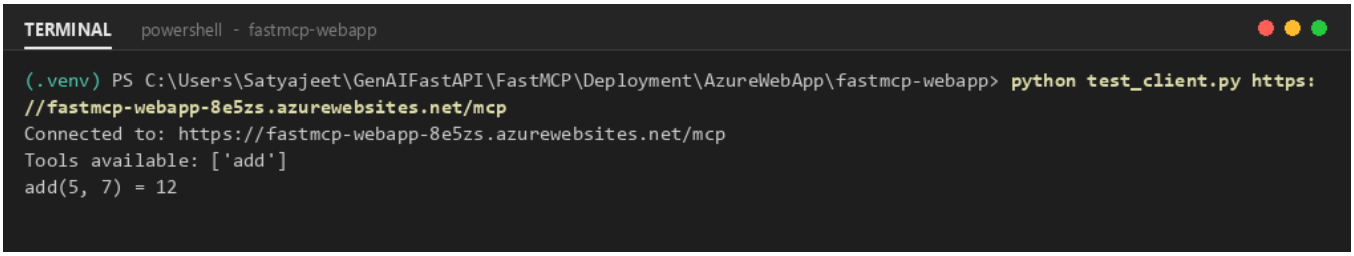



Figure 11.1 - The deployed server answers over HTTPS

## Connecting to Microsoft Foundry

In a Foundry agent, add an MCP tool (Tools > Add > Model Context Protocol) with:

- **Server URL:** `https://fastmcp-webapp-8e5zs.azurewebsites.net/mcp`
- **Server label:** e.g. fastmcp-azure
- **Transport:** Streamable HTTP • **Authentication:** none
- **Try it:** ask the agent *"Use the add tool to add 25 and 17"* - it should answer 42.

**Cold start:** on the Free plan the app sleeps when idle, so the first call can take a few seconds. Call it once before class.

**Security:** this demo endpoint is public with no authentication. Before exposing real business tools, add authentication (e.g. App Service / Functions built-in authentication with Microsoft Entra ID, or API Management).

## Step 12: Clean up (avoid charges after the class)

```
az group delete --name $RG --yes --no-wait
```

Or in the portal: Resource groups > fastmcp-webapp-rg > **Delete resource group**. To re-deploy after editing server.py, repeat only Step 8.

# Appendix A: Complete source files

Copy these into the **fastmcp-webapp** folder. File names must match exactly (note the leading dot in `.dockerignore`).

### requirements.txt

```
fastmcp==4.1.3
uvicorn==0.54.0
```

### server.py

```
from fastmcp import FastMCP

mcp = FastMCP("Azure MCP Server")


@mcp.tool
def add(a: int, b: int) -> int:
    """Add two numbers."""
    return a + b


if __name__ == "__main__":
    mcp.run(
        transport="http",
        host="0.0.0.0",
        port=8000,
        path="/mcp"
    )
```

### test_client.py

```
import asyncio
import sys

from fastmcp import Client

URL = sys.argv[1] if len(sys.argv) > 1 else "http://localhost:8000/mcp"


async def main():
    async with Client(URL) as client:
        tools = await client.list_tools()
        print("Connected to:", URL)
        print("Tools available:", [tool.name for tool in tools])

        result = await client.call_tool("add", {"a": 5, "b": 7})
        print("add(5, 7) =", result.data)


asyncio.run(main())
```